Segmentation

Week 8 — Session 2: Watershed, components & evaluation

Practical Image Processing

Lecture 8.2 ▪ 60–70 min

Recap of Session 1

- **Thresholds** split by intensity: global Otsu (even light), adaptive (shadows).
- **Region** methods split by similarity: growing from seeds, split-and-merge, k-means colour.
- All of these still leave *touching* objects fused into one blob.

Today (Session 2)

Separate touching objects, label each one, and measure how good a segmentation is — then ship the week's deliverable.

Roadmap

The touching-objects problem

Watershed

Connected components

Evaluating a segmentation

Beyond classical

DocScan pipeline

Wrap-up

The touching-objects problem

The touching-objects problem

- Threshold + morphology gives blobs — but two touching digits become *one* blob.
- Counting/reading needs them separated. Erosion helps a little, but can break thin strokes.
- **Watershed** solves it properly, using the shape (a distance transform).

Key idea

The hard part of segmentation is rarely object-vs-background — it is object-vs-*object* when they touch.

Why erosion alone is not enough

Two touching blobs (1 = foreground):

0	1	1	1	1	0
0	1	1	1	1	0

0,1,1,1,1,0

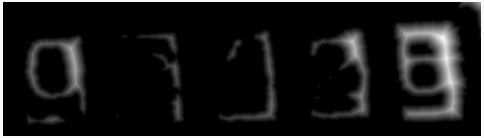
No gap exists — one connected component.

✕ Pitfall

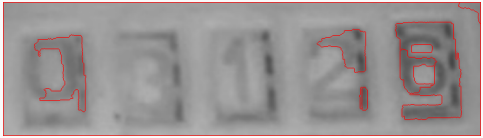
Erode until they separate and you also thin real strokes; erode less and they stay merged. There is no single erosion count that works for every pair.

Watershed

Distance transform → watershed



distance transform



watershed boundaries (red)

The distance transform peaks at each object's centre; watershed "floods" from those peaks and builds a ridge where floods meet — the separating boundary.

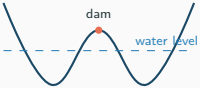
Distance transform — worked

Distance-to-nearest-background of the touching blob from before:

0	1	1	1	1	0
0	1	2	2	1	0
0	1	1	1	1	0

- Each foreground pixel gets its distance to the nearest 0.
- Two peaks appear (value 2) — one per digit centre — even though the shapes touch.
- Those peaks become the watershed **markers**: two basins, so two objects.

Watershed intuition



- Treat intensity (or −distance) as a landscape; "water" rises from marked minima.
- Where two catchment basins meet, build a dam — that dam is the object boundary.
- **Markers** (seeds) control it, or it **over-segments**.

The over-segmentation trap

✗ Pitfall

Raw watershed makes *every* local minimum its own basin — noise creates dozens of tiny bogus regions.

✓ Good practice

Marker-controlled watershed: flood only from *chosen* markers (sure-foreground) and let everything else be decided by the ridges.

Key idea

Never run watershed unseeded on a noisy image — always give it markers.

Marker-controlled watershed — the recipe

1. Threshold to a clean binary (Session 1).
2. Distance transform; its high peaks = **sure foreground**.
3. Dilate the binary = **sure background**; the rest is **unknown**.
4. Label the sure-foreground blobs (connected components) as markers.
5. Run `cv2.watershed`; it assigns every unknown pixel and marks ridges $= -1$.

✔ Good practice

Threshold on the distance peak (e.g. $0.5 \times \max$) trades off: higher $\rightarrow$ fewer, surer markers; lower $\rightarrow$ risk of re-merging.

Watershed in code (marker-controlled)

```
dist = cv2.distanceTransform(binary, cv2.DIST_L2, 5)
_, sure_fg = cv2.threshold(dist, 0.5*dist.max(), 255, 0)  # digit centres
sure_bg = cv2.dilate(binary, k, iterations=3)
unknown = cv2.subtract(sure_bg, np.uint8(sure_fg))
_, markers = cv2.connectedComponents(np.uint8(sure_fg))
markers = markers + 1; markers[unknown==255] = 0
markers = cv2.watershed(color_img, markers)               # -1 = boundaries
```

- `markers+1` keeps background $= 1$; unknown set to 0 for watershed to fill.

Watershed pitfalls

- Peak threshold too high $\rightarrow$ a real digit gets no marker and merges with a neighbour.
- Peak threshold too low $\rightarrow$ one digit sprouts two markers and splits.
- Forgetting to pass the *colour* image to `cv2.watershed` (it needs 3 channels).

✕ Pitfall

The -1 ridge pixels belong to *no* object — remember to ignore or reassign them before measuring areas.

Connected components

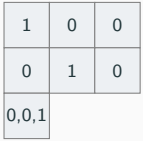
Labelling separate blobs



each blob a different colour

- **Connected components** labels each separate white region with an integer.
- Once separated, each digit is its own component you can crop and read (Week 13).
- `cv2.connectedComponents` / `...WithStats` give labels + areas + boxes.

4- vs 8-connectivity



- **4-connectivity**: only up/down/left/right $\rightarrow$ 3 separate components.
- **8-connectivity**: diagonals count $\rightarrow$ 1 component.
- The choice changes counts — pick it deliberately.

✕ Pitfall

A diagonal "bridge" of noise can fuse two digits under 8-connectivity. For separated digits, 4-connectivity is often safer.

Two-pass labelling (classic algorithm)

1. **Pass 1:** scan row-major; give each foreground pixel the smallest label among its already-seen neighbours (or a new label). Record label equivalences.
2. **Resolve:** union-find collapses equivalent labels to one representative.
3. **Pass 2:** relabel every pixel with its representative.

Note

OpenCV's `connectedComponents` does this for you in $O(N)$; knowing the two passes explains why label numbers are not contiguous before resolution.

Components with stats — crop each digit

```
n, labels, stats, cents = cv2.connectedComponentsWithStats(
        binary, connectivity=8)
digits = []
for i in range(1, n):                       # 0 is background
    x, y, w, h, area = stats[i]
    if area < 40:                           # drop specks
        continue
    digits.append(binary[y:y+h, x:x+w])     # one cropped digit
digits.sort(key=lambda d: d.shape)          # (then order by x)
```

- `stats` gives x, y, w, h, area per label; `cents` the centroids.

Filtering & ordering components

- **Area filter**: drop components below a minimum area (noise) or above a maximum (the page border).
- **Aspect / fill**: a digit has a plausible height:width and fill ratio; use it to reject smudges.
- **Order**: sort surviving boxes by their x-centroid so digits read left-to-right.

✓ Good practice

Sorting by x (not by label id) is what makes "42" come out as `4,2` and never `2,4`.

Evaluating a segmentation

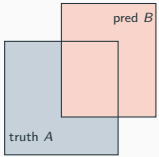
Evaluating a segmentation

- With ground-truth masks: **IoU** (intersection over union) and **Dice** score.
- Over-segmentation (too many regions) vs under-segmentation (merged regions) — both are errors.
- For DocScan: the real test is downstream — can each digit be read after segmentation?

Math

$\mathrm{IoU} = \dfrac{|A \cap B|}{|A \cup B|}, \quad \mathrm{Dice} = \dfrac{2|A \cap B|}{|A| + |B|}$ — both are 1.0 at perfect overlap.

IoU — a worked example



Truth $A = 3 \times 3 = 9$, pred $B = 2.5 \times 3 = 7.5$, overlap $= 1.5 \times 2 = 3$:

Math

$$\text{IoU} = \frac{3}{9 + 7.5 - 3} = \frac{3}{13.5} = 0.22$$

$$\text{Dice} = \frac{2 \cdot 3}{9 + 7.5} = \frac{6}{16.5} = 0.36$$

Dice is always $\geq$ IoU for the same masks.

Over- vs under-segmentation

✗ Under-segmentation

Two digits share one label. Downstream reads a single wrong "digit." Cause: marker missing / peak threshold too high.

✗ Over-segmentation

One digit split into two labels. Downstream reads a phantom extra digit. Cause: noise minima / peak threshold too low.

Key idea

Tune the distance-peak threshold to the count you expect: the right number of components is itself a segmentation check.

Beyond classical

Classical vs deep segmentation

- Classical (threshold/watershed/k-means): fast, interpretable, no training — great for clean, structured images.
- Deep (U-Net, Mask R-CNN, SAM): learns to segment complex scenes, but needs data/compute.
- For meters/documents, classical often suffices; deep shines on messy natural scenes (Week 13–14).

GrabCut & graph-based methods (brief)

- **GrabCut**: model foreground/background colour distributions and cut the graph optimally — interactive object cutout.
- Graph-cut / normalized-cut view segmentation as partitioning a pixel graph.
- These bridge classical and modern; good to know they exist.

Note

You draw one box around the object; GrabCut iterates colour models + graph cut to refine the boundary — no per-pixel labelling by hand.

DocScan pipeline

The segmentation pipeline for DocScan



- Threshold → clean → separate touching digits → label each → read.
- Each digit becomes an isolated image the CNN can classify.

Putting it together on the meter

1. Adaptive threshold the meter crop (shadow-robust, Session 1).
2. Morphological open to remove speckle (Week 7).
3. Distance transform + marker-controlled watershed to split touching digits.
4. `connectedComponentsWithStats`, area-filter, sort by x.
5. Emit one cropped, ordered image per digit.

✔ Good practice

Validate by counting: if the meter shows 5 digits, your pipeline should output exactly 5 components. Any other count flags a tuning problem.

Wrap-up

What you should now be able to do

- Separate touching objects with a marker-controlled watershed.
- Read a distance transform and place markers from its peaks.
- Label blobs with connected components; filter and order them.
- Evaluate a segmentation with IoU/Dice and diagnose over/under-segmentation.

This week's deliverable

Due before Week 9 (see `project/`)

1. Compare Otsu vs adaptive on meter images; pick the winner per lighting.
2. k-means colour segmentation of a car/plate photo at $k = 2, 4, 8$; discuss.
3. Watershed to separate touching meter digits; label each with connected components.
4. Report IoU/Dice against a hand-drawn mask for one image.
5. DocScan milestone: output one cropped image per digit, in order.

Segmentation turns pixels into objects.

Watershed separates touching shapes, components label them, IoU/Dice grade them.